

ECO-CITY PULSE

MASTER FUNCTIONALITY & REQUIREMENTS SPECIFICATION

Executive Summary

Eco-City Pulse is an end-to-end urban environmental intelligence platform designed to ingest, process, analyze, and predict urban environmental conditions. This project serves as a comprehensive B.Tech Computer Science/Data Science capstone, demonstrating mastery of the BACSE301 Exploratory Data Analysis syllabus while extending into a full-stack Machine Learning application.

The platform avoids being "just an ML model." Instead, it is a unified pipeline of Data Collection → Data Engineering → EDA → Statistical Analysis → Visualization → Feature Engineering → ML → Evaluation → Explainability → Prediction → Decision Support.

Problem Statement & Motivation

Urban environments suffer from rapidly fluctuating pollution levels, traffic congestion, and micro-climatic changes. Existing tools either provide raw, uncontextualized data (simple API dashboards) or opaque predictions (black-box ML models). There is a critical need for an interpretable platform that not only forecasts environmental conditions but strictly applies Exploratory Data Analysis (EDA) to uncover the *why* behind the predictions.

Motivation: To transition academic data science concepts—specifically missing value imputation, outlier detection, statistical distributions, dimensionality reduction, and predictive modeling—into a robust, interactive, real-world software engineering product.

Project Vision & Objectives

To build the most academically rigorous and technically robust version of a modular monolith data science platform. Eco-City Pulse will ingest heterogeneous data, harmonize it, expose it to

automated and manual EDA, and power an interpretable Machine Learning engine to predict PM2.5 levels and generate an "Environmental Stress Index."

1. Implement a resilient data ingestion and harmonization engine (API + CSV fallback).
2. Build an automated and interactive EDA studio demonstrating syllabus-prescribed statistical techniques.
3. Develop an interpretable ML pipeline predicting future PM2.5 concentrations.
4. Deliver a production-quality frontend for exploratory visualization and decision support.
5. Ensure strict data leakage prevention and robust MLOps practices.

Scope, Target Users & Personas

In Scope: Multi-source data ingestion, automated data cleaning (MICE, IQR/Z-score), feature engineering, automated EDA reports, dimensionality reduction (PCA), time-series analysis (ARIMA/Prophet baseline + XGBoost/Random Forest), spatial mapping, REST API backend, interactive frontend.

Out of Scope: Distributed streaming architecture (Kafka), Kubernetes clusters, real-time millisecond latency requirements, native mobile applications, causal inference (only correlation will be claimed).

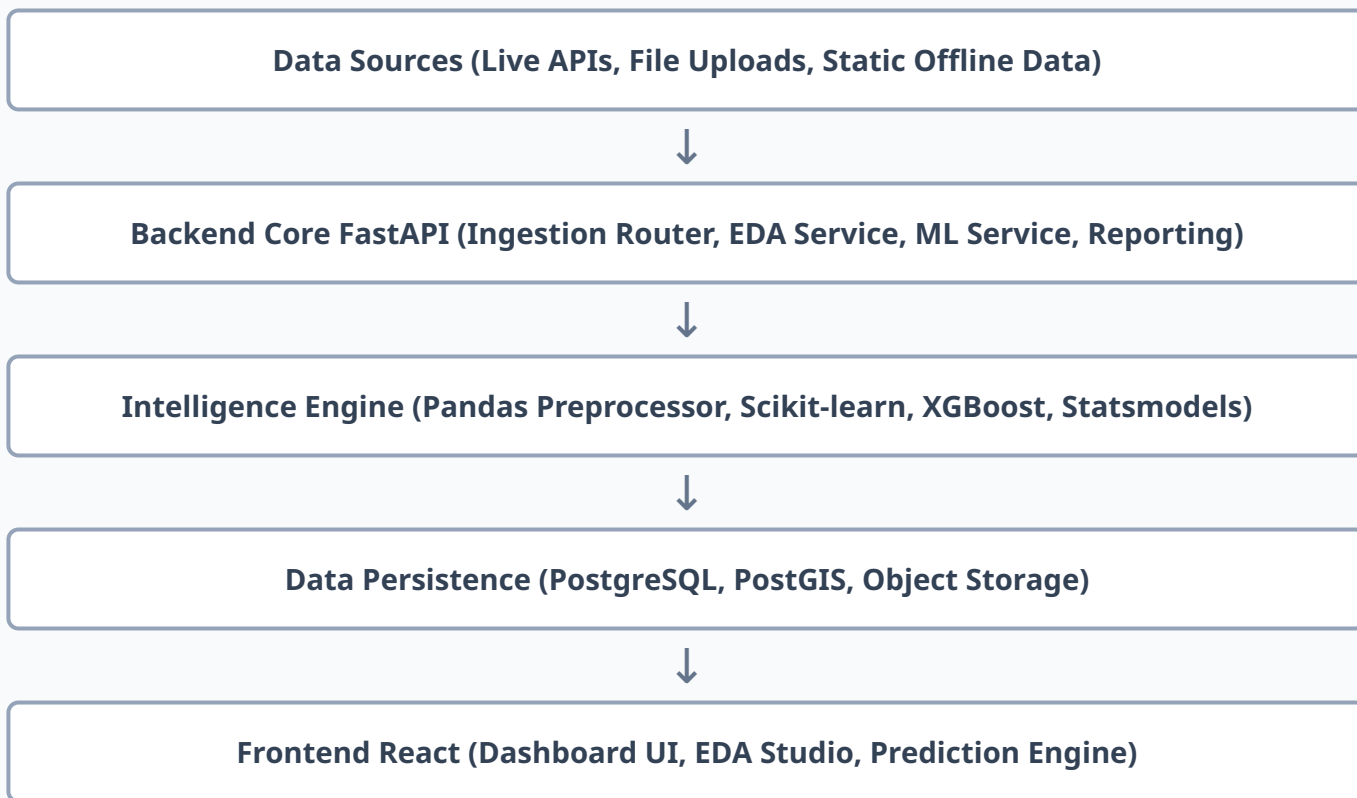
Role	Capabilities	Primary Persona
General User	View dashboards, check predictions, explore basic maps, read reports.	Alex, a concerned citizen planning outdoor activities based on air quality.
Analyst/Data Scientist	Run deep EDA, trigger preprocessing pipelines, evaluate metrics, configure SHAP.	Dr. Lin, an academic researching traffic impact on PM2.5.
Administrator	Manage API credentials, monitor ingestion logs, configure thresholds.	Sam, the system owner maintaining data freshness and backend health.

Architecture Overview

Functional Architecture

1. **Source Tier:** Live APIs (AQICN, OpenWeather), CSV/JSON uploads, offline fallbacks.
2. **Ingestion & Quality Tier:** Validation, schema normalization, missingness detection.
3. **Intelligence Tier:** EDA engine, Statistical Engine, Feature Store, ML Pipeline.
4. **Service Tier:** FastAPI REST endpoints, caching, report generation.
5. **Presentation Tier:** React SPA, interactive charts, prediction forms.

System Flow



Deployment Architecture

Modular Monolith via Docker Compose.

- **Container 1:** React/Vite Frontend (Nginx).
- **Container 2:** FastAPI Python Backend.
- **Container 3:** PostgreSQL + PostGIS extension.

Data Architecture & Management

Data Sources

- **Air Quality:** AQI, PM2.5, PM10, NO2, SO2, CO, O3. (Source: AQICN API / Kaggle Historical)
- **Weather:** Temperature, Humidity, Wind Speed, Wind Direction, Precipitation. (Source: OpenWeather API)
- **Traffic (Proxy):** Traffic density score or congestion indices. (Source: TomTom API or synthetic baseline)
- **Spatial Boundaries:** GeoJSON of city districts.

Data Ingestion & Harmonization

The system MUST operate without live APIs if necessary using Scheduled (Cron), Manual (Admin trigger), Upload (CSV), or Demo (Pre-loaded DB) modes. Timestamps are converted universally to UTC. Locations are standardized to Decimal Degrees. Data is resampled to hourly granularity for time-series alignment.

Data Quality Engine (Module 2 Syllabus)

- **Missing Values:** Detect MCAR, MAR, MNAR. Resolve using MICE (Multivariate Imputation by Chained Equations) for complex features, Forward/Backward fill for contiguous gaps.
- **Outliers:** Detect using IQR (Boxplot bounds), Z-Score (>3), Isolation Forest. Action: Flag as `is_anomaly`. Do *not* auto-delete (could be a valid pollution spike).

Intelligence & Analysis

Feature Engineering

- **Temporal:** `hour_of_day`, `day_of_week`, `is_weekend`, `month`, `season`.
- **Lag Features:** `PM2.5_lag_1h`, `PM2.5_lag_24h`, `Temp_lag_3h`.
- **Rolling Stats:** `PM2.5_rolling_mean_24h`, `traffic_rolling_std_6h`.
- **Transformations:** Log transformation on highly skewed pollutants (e.g., CO, SO2).

Dimensionality Reduction & ESI

PCA (Principal Component Analysis): Used to construct the **Environmental Stress Index (ESI)**. Standardize continuous variables, extract Principal Component 1 (PC1), and normalize the score from 0-100. **t-SNE:** Used exclusively in the EDA Studio for 2D clustering visualization.

Machine Learning Pipeline

Target: Predict continuous PM2.5 levels 1, 6, and 24 hours into the future (Regression). Models evaluated include Naive Lag-1, Ridge Regression, Random Forest, and XGBoost.

Raw Data → Impute/Clean → Time-Aware Split → Feature Selection → Scale/Encode → Train Models → Cross-Validation → Model Registry

Leakage Safeguards: Strict temporal splitting. Scalers fit *only* on the training set. Lag features computed *before* splitting, but target variable shifted correctly.

Core Feature Specifications

Feature ID	Feature Name	Inputs	Processing / Outputs	Priority
FEAT-01	Ingestion Engine	API configs, CSV	Validate schema, normalize, write to DB. Output: Ingestion log.	Must Have
FEAT-02	Automated Profile	Dataset ID	Calculates mean, median, IQR, missingness. Output: JSON stats.	Must Have
FEAT-03	MICE Imputation	Raw DataFrame	Applies IterativeImputer. Output: Clean DataFrame.	Must Have
FEAT-04	ESI Generator	Scaled Data	Applies PCA, extracts PC1. Output: 0-100 Score.	Should Have
FEAT-05	Model Trainer	Target, Features	Time-split, train XGBoost, log metrics. Output: Model artifact.	Must Have

Feature ID	Feature Name	Inputs	Processing / Outputs	Priority
FEAT-06	Predict Endpoint	Location, Time	Fetches lag features, runs inference. Output: Predicted PM2.5.	Must Have

API Specification (Backend Base URL: /api/v1)

- GET /data/sources - List data sources and ingestion health.
- POST /eda/profile - Generate univariate and bivariate statistics.
- POST /eda/reduce - Perform Dimensionality Reduction (PCA).
- POST /ml/predict - Request a PM2.5 prediction.

```
// Example POST /ml/predict Response
{
  "prediction": 45.2,
  "unit": "ug/m3",
  "confidence_interval": [38.5, 51.9],
  "top_features": {"lag_24_pm25": 0.45, "wind_speed": 0.22}
}
```

Database Design (PostgreSQL)

- data_sources: id, name, api_url, status, last_run.
- observations: id, source_id, timestamp, lat, lon, pm25, pm10, temp, humidity, traffic_score, is_anomaly.
- models: id, name, target, features_used, mae, rmse, r2, created_at, artifact_path.
- predictions: id, model_id, target_time, predicted_value, actual_value.

Frontend UX Architecture

Stack: React, TypeScript, Vite, Tailwind CSS, Plotly.js, React-Leaflet.

- **Overview Dashboard:** Hero KPIs (Current ESI, PM2.5). Leaflet map showing spatial pollution gradients. 24-hour prediction trendline.
- **EDA Studio:** Missingness matrix, Plotly histograms, interactive correlation heatmap, parallel coordinates, STL decomposition overlays.
- **Model Lab:** Table of trained models, hyperparameter configurations, metrics, and feature importance bar charts.

Security, Privacy & Ethics

API requests validated via Pydantic. Environment variables for secrets. CORS restricted to frontend origin. Only public environmental data utilized. Text analysis on social data strictly aggregates metrics (no PII). The UI explicitly states: *"Predictions rely on sensor placement which may exhibit geographic/socioeconomic bias. Correlation shown does not equal causation."*

Project Directory Structure

```
eco-city-pulse/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── services/
│   │       └── App.tsx
│   ├── package.json
│   └── tailwind.config.js
├── backend/
│   ├── api/
│   │   ├── routes/
│   │   └── dependencies.py
│   ├── core/
│   │   ├── config.py
│   │   └── exceptions.py
│   ├── services/
│   │   ├── eda_service.py
│   │   ├── ml_service.py
│   │   └── ingestion_service.py
│   ├── tests/
│   ├── main.py
│   └── requirements.txt
├── data/
│   ├── raw/
│   └── processed/
├── docker-compose.yml
└── README.md
```

Academic / Syllabus Mapping (BACSE301)

Module	Topic	Eco-City Pulse Implementation
Mod 1	Data Collection & Structure	Multi-source API & CSV ingestion, JSON validation, DB storage.
Mod 2	Data Preprocessing	MICE imputation, Z-score/Isolation Forest anomaly detection, scaling.

Module	Topic	Eco-City Pulse Implementation
Mod 3	Descriptive Stats & Vis	Automated EDA dashboard (histograms, boxplots, correlation heatmaps).
Mod 4	Dimensionality & Time-Series	PCA for Environmental Stress Index, STL Decomposition for temporal trends.
Mod 5	Advanced Visualization	Parallel coordinates, missingness matrix, automated HTML/PDF reports.